

MERN STACK

E-Commerce Web App

Roman Urdu Mein — Zero Se Hero Tak

CHAPTER 3

INTERMEDIATE

THEORY + CODE

ROMAN URDU

CHAPTER 3

MongoDB & Mongoose — Real Database!

Data permanently store karo — asli websites wali database!

Is Chapter Mein Aap Seekhenge:

- Database kya hai aur SQL vs NoSQL ka farq
- MongoDB kya hai aur kaise kaam karta hai
- MongoDB Atlas — free cloud database setup karna
- Express server ko MongoDB se connect karna
- Mongoose Schema aur Model banana
- CRUD — Create, Read, Update, Delete with real DB
- Data Validation — mongoose ke saath
- Queries — filter, sort, paginate karna



ShopEase ke saare Models banana

■ Chapter 1 & 2 Recap

Node.js, npm, Express routes, middleware, JSON — yeh sab aana chahiye. Chapter 3 mein hum in sab ko real database se connect karenge!

TABLE OF CONTENTS

1.0 Database Kya Hai?

- 1.1 SQL vs NoSQL
- 1.2 MongoDB ka structure
- 1.3 Document kya hota hai?

2.0 MongoDB Atlas Setup

- 2.1 Free account banana
- 2.2 Cluster create karna
- 2.3 Connection string lena

3.0 Project Ko DB Se Connect Karo

- 3.1 mongoose install karna
- 3.2 .env file — secrets protect karo
- 3.3 DB connection code
- 3.4 dotenv setup

4.0 Mongoose Schema & Model

- 4.1 Schema kya hota hai?
- 4.2 Schema data types
- 4.3 Model banana
- 4.4 Model ka kaam

5.0 ShopEase Models Banana

- 5.1 Product Model
- 5.2 User Model
- 5.3 Order Model
- 5.4 Models ko server.js se import karna

6.0 CRUD with Real Database

- 6.1 CREATE — .create() / .save()
- 6.2 READ — .find() / .findById()
- 6.3 UPDATE — .findByIdAndUpdate()
- 6.4 DELETE — .findByIdAndDelete()

7.0 Validation aur Defaults

- 7.1 Required fields
- 7.2 Default values

- 7.3 Custom validators
- 7.4 Mongoose error handling

8.0 Advanced Queries

- 8.1 Filter — `.find({category:'shoes'})`
- 8.2 Sort — `.sort({price:-1})`
- 8.3 Pagination — `.skip().limit()`
- 8.4 Select fields — `.select()`

9.0 Summary & Practice

- 9.1 Chapter recap
- 9.2 Practice exercises
- 9.3 Chapter 4 preview

SECTION 1 — Database Kya Hai?

1.0 — Database Kya Hai?

Ab tak humara data server mein ek array mein tha — server band karo, sab khatam! Real websites mein data **database** mein store hota hai — permanently. Jaise ghar mein ek badi almari hoti hai jisme saari cheezein organized rakhte hain — database wahi almari hai digital data ke liye.

Bina database ke kya hota tha humara server mein:

Masla	Asar
Server restart	Saara data khatam — products, users, orders sab gone!
Multiple servers	Har server ka alag data — inconsistency
Data search karna	Array mein manually dhundhna — slow aur limited
Data relationships	User ka order kaun sa? — array mein manage karna mushkil

1.1 — SQL vs NoSQL — Kya Farq Hai?

Databases do tarah ke hote hain. MongoDB **NoSQL** type ka hai:

Feature	SQL (PostgreSQL, MySQL)	NoSQL (MongoDB) ■ Hum yeh use karenge
Data format	Tables — rows aur columns	Documents — JSON jaisi objects
Schema	Fixed — pehle se define karna padta	Flexible — baad mein change ho sakta
Relations	JOIN queries — complicated	Embedded documents ya references
Scale karna	Vertical — bada server chahiye	Horizontal — zyada servers add karo
Seekhna	SQL language seekhni padti	JSON jaisi syntax — JS developers ke liye asaan
Speed	Complex queries slow ho sakti hain	Simple queries bohat fast hain

1.2 — MongoDB Ka Structure

MongoDB mein data hierarchy kuch aisi hoti hai:

■	Database	shopease_db	Poora project ka ek database — jaise ek ghar
---	----------	-------------	--

■	Collection	<code>products, users</code>	Related documents ka group — jaise almari ka drawer
■	Document	<code>{ name: 'Shoes' }</code>	Ek record — jaise ek file — JSON format mein
■	Field	<code>name, price, _id</code>	Document ki ek property — jaise ek column

1.3 — MongoDB Document Kaisa Dikhta Hai?

Ek real product document MongoDB mein kuch aisa hoga:

```
{
  "_id": ObjectId("64f8a3b2c1d2e3f4a5b6c7d8"),
  "name": "Nike Air Max 270",
  "price": 15000,
  "category": "Footwear",
  "description": "Bohat comfortable running shoes",
  "images": ["img1.jpg", "img2.jpg", "img3.jpg"],
  "rating": { "average": 4.5, "numReviews": 128 },
  "stock": 50,
  "isFeatured": true,
  "createdAt": ISODate("2024-01-15T10:30:00Z")
}
```

■ TIP

`_id` MongoDB khud automatically banata hai — har document ka unique ID. Yeh `ObjectId` type ka hota hai — 24 character ka hex string. Tumhe khud ID banana nahi padta!

SECTION 2 — MongoDB Atlas Setup

2.0 — MongoDB Atlas Kya Hai?

MongoDB Atlas MongoDB ka official free cloud service hai. Database tumhare computer pe nahi, internet pe hoga — jahan se bhi access kar sako. Free tier mein 512MB storage milta hai — jo humari practice ke liye kaafi hai!

2.1 — Atlas Account banana (Step by Step)

01	mongodb.com/atlas pe jao	Browser mein yeh URL likho aur Enter dabao
02	'Try Free' button click karo	Registration form aayega — email se sign up karo
03	Free plan choose karo	'M0 Free' option select karo — credit card nahi chahiye!
04	Cloud provider choose karo	AWS ya Google Cloud — koi bhi, region nearest choose karo
05	Cluster ka naam do	Default 'Cluster0' theek hai — ya apna naam likho
06	Create Deployment click karo	2-3 minutes lagenge cluster banane mein — wait karo

2.2 — Database User aur Network Access

Cluster banne ke baad 2 zaroori settings karo:

Step A	Database User banana	Left sidebar: Security -> Database Access -> Add New User. Username aur strong password daalo. Role: 'Atlas admin' choose karo. Password yaad rakhna — connection string mein lagega!
Step B	Network Access allow karna	Left sidebar: Security -> Network Access -> Add IP Address. 'Allow Access from Anywhere' click karo (0.0.0.0/0) — Development mein yeh theek hai. Production mein specific IP daalo.

2.3 — Connection String Lena

Connection string woh address hai jisse humara Node.js server MongoDB se milega:

- Cluster page pe 'Connect' button click karo
- 'Drivers' option choose karo
- 'Node.js' select karo, version latest
- Connection string copy karo — kuch aisa dikhega:

```
mongodb+srv://yourUsername:yourPassword@cluster0.abcde.mongodb.net/shop  
ease?retryWrites=true&w=majority
```

```
# Yahan yeh replace karo:
```

```
# yourUsername -> jo username banaya tha
```

```
# yourPassword -> jo password banaya tha
```

```
# shopease -> database ka naam (khud likho)
```

■ YAAD RAHO

Connection string mein password likha hota hai — ise kabhi GitHub pe mat daalo! Hum ise .env file mein rakhenge — .gitignore mein add karo. Yeh security ka rule number 1 hai!

SECTION 3 — Project Ko Database Se Connect Karo

3.0 — Mongoose Kya Hai?

Mongoose ek ODM (Object Document Mapper) library hai. Yeh MongoDB aur Node.js ke beech mein ek layer hai jo kaam asaan karta hai. MongoDB directly use karna complicated hai — Mongoose ne sab simple kar diya:

Feature	MongoDB Driver (directly)	Mongoose ■
Schema define karna	Nahi hota	Haan — data structure fix hoti hai
Validation	Khud likhni padti	Built-in validation
Relationships	Manually handle karo	.populate() se asaan
Query methods	Raw MongoDB syntax	Simple JS-like methods
Error messages	Raw MongoDB errors	Human-readable errors

3.1 — Packages Install Karo

```
# Mongoose install karo

npm install mongoose

# dotenv install karo — .env file read karne ke liye

npm install dotenv

# Check karo package.json mein aa gaye:

# "dependencies": {
#   "express": "^4.18.2",
#   "mongoose": "^8.0.0",
#   "dotenv": "^16.0.0",
#   "cors": "^2.8.5"
# }
```

3.2 — .env File Banana — Secrets Protect Karo!

.env file mein hum sensitive information rakhte hain — database password, secret keys, etc. Yeh file code mein hardcode nahi karte aur GitHub pe nahi jaati.

Project root mein **.env** naam ki file banao:

```
# .env file — (root mein, server.js ke saath)

# MongoDB connection string
MONGO_URI=mongodb+srv://yourUsername:yourPassword@cluster0.abcde.mongodb.net/shopease

# Server port
PORT=5000

# Environment
NODE_ENV=development
```

```
# .gitignore mein add karo (ZAROOR!):

node_modules/

.env
```

■ YAAD RAHO

.env file ko **.gitignore** mein add karna ZAROOR hai — warna password GitHub pe public ho jaayega. Yeh ek bahut common aur dangerous mistake hai!

3.3 — Database Connection File Banana

Ek alag file banao: **config/db.js** — saari connection logic yahan:

```
// config/db.js

const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    // .env se MONGO_URI lo
    const conn = await mongoose.connect(process.env.MONGO_URI);

    console.log(`MongoDB Connected! Host: ${conn.connection.host}`);
  } catch (error) {
    // Connection fail hua
    console.error(`DB Error: ${error.message}`);
  }
}
```

3.4 — server.js Update Karo

Ab server.js mein database connect karo:

[illegible]

3.5 — Key Lines Ka Matlab

<code>require('dotenv').config()</code>	Sab se pehle yeh call karo — .env file padhta hai aur sari values process.env mein daal deta hai. ZAROOR pehle likhna!
<code>process.env.MONGO_URI</code>	process.env se .env file ki values access karte hain. MONGO_URI .env mein jo naam diya wahi likhna.
<code>process.env.PORT 5000</code>	Agar PORT .env mein hai toh woh use karo, warna 5000 default. matlab 'ya phir'.
<code>connectDB()</code>	Database connection start karo — async function hai isliye errors try-catch mein handle hote hain.
<code>process.exit(1)</code>	Connection fail hone pe server band karo. 1 matlab error ke saath exit — sab sahi hone pe 0 hota.
<code>module.exports = connectDB</code>	Is function ko doosri files mein use karne ke liye export karo — require() se import hoga.

■ SAHI KIYA

npm run dev chalao — console mein 'MongoDB Connected!' dikhega. Agar nahi dikha toh connection string check karo — username/password sahi hai? Network Access mein IP allow hai?

SECTION 4 — Mongoose Schema & Model

4.0 — Schema Kya Hota Hai?

Schema ek **blueprint** hai — jaise ghar ka naksha. Naksha batata hai ke kamare kahan honge, kitne bade honge. Schema batata hai ke document mein kaun se fields honge, unka type kya hoga, kya required hai, default kya hai.

Model ek **class** hai jo schema se bana hota hai. Model se hum database mein documents create, read, update, delete karte hain. Schema = naksha | Model = contractor jo naksha follow karta hai.

4.1 — Schema Data Types

Type	Example	Kab Use Karo?
String	'Nike Shoes'	Text — naam, description, email, address
Number	5000, 4.5, -10	Sankhya — price, rating, quantity, age
Boolean	true / false	Haan/Nahi — isActive, isFeatured, inStock
Date	new Date()	Wagt — createdAt, updatedAt, dob
ObjectId	mongoose.Schema.Types.ObjectId	Reference — doosre model ki ID
Array	[String], [Number]	List — images, tags, sizes
Mixed	Schema.Types.Mixed	Koi bhi type — flexible (use carefully)
Map	Map	Key-value pairs — dynamic fields

4.2 — Simple Schema aur Model Banana

Ek simple example — pehle samjho, phir ShopEase wale models dekhenge:

```
const mongoose = require('mongoose');

// Step 1: Schema define karo

const productSchema = new mongoose.Schema({
  name: { type: String, required: true },
  price: { type: Number, required: true },
```

```

    stock: { type: Number, default: 0 },
    active: { type: Boolean, default: true },
  }, {
    timestamps: true // createdAt aur updatedAt auto add hoga
  });

// Step 2: Model banana — Schema se
// 'Product' = model naam, MongoDB mein 'products' collection banega
const Product = mongoose.model('Product', productSchema);

// Step 3: Export karo
module.exports = Product;

```

4.3 — Schema Options Ka Matlab

<code>type: String</code>	Field ka data type — String, Number, Boolean, Date, ObjectId, etc.
<code>required: true</code>	Yeh field zaroori hai — agar nahi di toh validation error aayega.
<code>default: 0</code>	Agar value nahi di toh automatically yeh value use hogi.
<code>timestamps: true</code>	Schema option — createdAt aur updatedAt fields automatically add aur update hoti hain.
<code>mongoose.model()</code>	Schema se Model banata hai. Pehla argument collection naam (plural lowercase) determine karta hai.

SECTION 5 — ShopEase Ke Models Banana

5.0 — Models Ka Plan

ShopEase mein humein 3 main models chahiye. Har model ek alag file mein hoga: **models/** folder banao aur yahan sab files daalo.

Model	File	Collection	Kya Store Karta Hai?
Product	models/Product.js	products	Saare products — naam, price, images, stock
User	models/User.js	users	Users — naam, email, hashed password, address
Order	models/Order.js	orders	Orders — user, products, total, status, payment

5.1 — Product Model (models/Product.js)

```
// models/Product.js

const mongoose = require('mongoose');

const reviewSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  name: { type: String, required: true },
  rating: { type: Number, required: true, min: 1, max: 5 },
  comment: { type: String, required: true },
}, { timestamps: true });

const productSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Product naam zaroori hai!'],
    trim: true, // extra spaces remove karo
    maxLength: [200, 'Naam 200 se zyada characters nahi ho sakta']
  },
  description: {
    type: String,
    required: [true, 'Description zaroori hai!'],
```

```

    },
    price: {
      type: Number,
      required:[true, 'Price zaroori hai!'],
      min: [0, 'Price 0 se kam nahi ho sakta']
    },
    images: [{ type: String }], // Array of image URLs
    category: {
      type: String,
      required: true,
      enum: ['Footwear', 'Clothing', 'Electronics', 'Books', 'Bags', 'Other']
    },
    stock: { type: Number, default: 0, min: 0 },
    isFeatured: { type: Boolean, default: false },
    reviews: [reviewSchema],
    rating: { type: Number, default: 0 },
    numReviews: { type: Number, default: 0 },
    seller: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User' // User model se reference
    }
  }, { timestamps: true });

module.exports = mongoose.model('Product', productSchema);

```

5.2 — User Model (models/User.js)

```

// models/User.js

const mongoose = require('mongoose');
const bcrypt = require('bcryptjs'); // Chapter 5 mein install karen ge

const userSchema = new mongoose.Schema({
  name: { type: String, required: [true, 'Naam zaroori hai!'], trim: true },
  email: {

```

```

    type: String,
    required: [true, 'Email zaroori hai!'],
    unique: true, // Duplicate emails nahi challenge
    lowercase: true, // Hamesha lowercase store karo
    match: [/^[a-zA-Z0-9_]+@[a-zA-Z0-9_]+\.[a-zA-Z]{2,4}$/, 'Valid email daalo!']
  },
  password: {
    type: String,
    required: [true, 'Password zaroori hai!'],
    minLength: [6, 'Password 6+ characters ka hona chahiye'],
    select: false // By default password query mein nahi aayega
  },
  role: { type: String, enum: ['user', 'admin'], default: 'user' },
  avatar: { type: String, default: 'default-avatar.jpg' },
  address: {
    street: String,
    city: String,
    state: String,
    zipCode: String,
  },
  isActive: { type: Boolean, default: true },
}, { timestamps: true });

// Password save karne se pehle automatically hash karo
userSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next(); // Sirf tab hash kar
  o jab badla ho
  this.password = await bcrypt.hash(this.password, 12);
  next();
});

module.exports = mongoose.model('User', userSchema);

```

■ TIP

User model mein password ka select: false hai — yeh bahut important security feature hai. Jab bhi users query karoge, password automatically nahi aayega. Explicitly chahiye toh .select('+password') use karo.

5.3 — Order Model (models/Order.js)

```
// models/Order.js

const mongoose = require('mongoose');

const orderItemSchema = new mongoose.Schema({
  product: { type: mongoose.Schema.Types.ObjectId, ref: 'Product', required: true },
  name: { type: String, required: true },
  image: { type: String, required: true },
  price: { type: Number, required: true },
  quantity: { type: Number, required: true, min: 1 },
});

const orderSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  orderItems: [orderItemSchema],
  shippingAddress: {
    address: { type: String, required: true },
    city: { type: String, required: true },
    zipCode: { type: String, required: true },
  },
  paymentMethod: { type: String, required: true, default: 'COD' },
  paymentResult: {
    id: String,
    status: String,
    updateTime: String,
  },
  itemsPrice: { type: Number, required: true, default: 0 },
  taxPrice: { type: Number, required: true, default: 0 },
  shippingPrice: { type: Number, required: true, default: 0 },
});
```

```
totalPrice:    { type: Number, required: true, default: 0 },
orderStatus: {
  type:    String,
  enum:    ['Processing', 'Shipped', 'Delivered', 'Cancelled'],
  default: 'Processing'
},
isPaid:        { type: Boolean, default: false },
paidAt:        Date,
isDelivered:   { type: Boolean, default: false },
deliveredAt:   Date,
}, { timestamps: true });

module.exports = mongoose.model('Order', orderSchema);
```

SECTION 6 — CRUD with Real Database

6.0 — Ab Real Database Se Kaam!

Chapter 2 mein hum array se kaam kar rahe the. Ab woh sab hatao — real MongoDB se data aayega. Har operation ke liye Mongoose ki methods hain:

Operation	Mongoose Method	SQL Equivalent
CREATE	<code>.create()</code> ya <code>.save()</code>	INSERT INTO
READ All	<code>.find()</code>	SELECT * FROM
READ One	<code>.findById()</code> ya <code>.findOne()</code>	SELECT WHERE id=
UPDATE	<code>.findByIdAndUpdate()</code>	UPDATE WHERE id=
DELETE	<code>.findByIdAndDelete()</code>	DELETE WHERE id=

6.1 — Routes File Banana (routes/productRoutes.js)

Pehle ek organized structure banao — routes alag file mein:

[illegible]

[illegible]

[illegible]

Aur server.js mein routes register karo:

```
// server.js mein add karo:

const productRoutes = require('./routes/productRoutes');

// Routes use karo

app.use('/api/products', productRoutes);

// Ab /api/products pe request aayegi toh productRoutes handle karega
```

6.2 — Key Methods Ka Matlab

<code>await Product.find()</code>	Database se saare products lao. await isliye ke yeh async operation hai — result aane mein waqt lagta hai.
<code>await Product.findById(id)</code>	ID se ek specific product dhundho. Agar nahi mila toh null return karta hai.
<code>await Product.create(data)</code>	Naya document database mein create karo. Validation automatically run hoti hai.
<code>findByIdAndUpdate(id, data, options)</code>	ID se document dhundho aur update karo. { new: true } updated document return karta hai.

<pre>{ runValidators: true }</pre>	Update karte waqt bhi schema validation run karo — default mein update pe validation nahi hoti!
<pre>await Product.findByIdAndDelete(id)</pre>	ID se document dhundho aur permanently delete karo.

SECTION 7 — Validation aur Defaults

7.0 — Validation Kyun Zaroori Hai?

Socho koi user ne price ki jagah 'abc' likhh diya — ya email nahi diya — ya negative price dali. Bina validation ke yeh sab database mein chala jaayega! Mongoose validation yeh sab check karta hai **data save hone se pehle**.

7.1 — Built-in Validators

```
const schema = new mongoose.Schema({

  // required — field zaroori hai

  name: { type: String, required: true },

  // Custom error message ke saath:

  name: { type: String, required: [true, 'Naam daalna zaroori hai!'] },

  // min / max — Number ke liye

  price: { type: Number, min: [0, 'Price negative nahi hoti!'], max: 100000 },

  // minLength / maxLength — String ke liye

  password: { type: String, minLength: [6, '6+ chars chahiye'], maxLength: 100 },

  // enum — sirf yeh values allowed hain

  role: { type: String, enum: ['user', 'admin'], default: 'user' },

  // unique — duplicate nahi aana chahiye

  email: { type: String, unique: true },

  // match — regex pattern

  email: { type: String, match: [/^S+@S+\.S+/, 'Valid email chahiye!'] },

  // trim — extra spaces remove karo

  name: { type: String, trim: true },

  // lowercase / uppercase

  email: { type: String, lowercase: true },

});
```

7.2 — Custom Validator Banana

Built-in validators enough nahi toh khud validator likho:

```
price: {
  type: Number,
  validate: {
    validator: function(v) {
      // true return karo agar valid, false agar invalid
      return v > 0 && v <= 100000000;
    },
    message: props => `${props.value} valid price nahi – 0 se bada hona chahiye!`
  }
},

// Phone number validator
phone: {
  type: String,
  validate: {
    validator: function(v) {
      return /^[0-9]{11}$/.test(v); // 11 digits
    },
    message: '11 digit ka phone number chahiye!'
  }
},
```

7.3 — Validation Errors Handle Karna

Validation fail hone pe Mongoose ek specific error throw karta hai. Ise properly handle karo:

```
router.post('/api/products', async (req, res) => {
  try {
    const product = await Product.create(req.body);
    res.status(201).json({ success: true, data: product });
  } catch (error) {
```

```

// Mongoose validation error
if (error.name === 'ValidationError') {
  const messages = Object.values(error.errors).map(e => e.message);
  return res.status(400).json({
    success: false,
    message: 'Validation failed!',
    errors: messages // ['Naam zaroori hai!', 'Price negative nahi
hoti!']
  });
}

// Duplicate key error (unique constraint)
if (error.code === 11000) {
  return res.status(400).json({
    success: false,
    message: 'Yeh email pehle se registered hai!'
  });
}

res.status(500).json({ success: false, message: 'Server error!' });
}
});

```

SECTION 8 — Advanced Queries

8.0 — Queries Ka Power!

Sirf `.find()` se kaam nahi chalta real app mein. User category filter karna chahta hai, price se sort karna chahta hai, page 2 dekhna chahta hai. Mongoose mein yeh sab bohat asaan hai:

8.1 — Filtering

```
// Sirf 'Footwear' category ke products

const products = await Product.find({ category: 'Footwear' });

// Price 1000 se zyada wale

const products = await Product.find({ price: { $gt: 1000 } });

// $gt = greater than | $gte = greater than or equal
// $lt = less than | $lte = less than or equal
// $ne = not equal | $in = in array

// Multiple conditions – AND

const products = await Product.find({
  category: 'Footwear',
  price: { $gte: 1000, $lte: 10000 },
  stock: { $gt: 0 } // In stock wale
});

// Search by name – case-insensitive

const query = 'nike';

const products = await Product.find({
  name: { $regex: query, $options: 'i' } // 'i' = case insensitive
});
```

8.2 — Sorting

```
// Price se sort karo – ascending (sasta pehle)

const products = await Product.find().sort({ price: 1 });

// Price se sort karo – descending (mahanga pehle)

const products = await Product.find().sort({ price: -1 });
```

```
// Rating descending, phir price ascending

const products = await Product.find().sort({ rating: -1, price: 1 });

// Naye pehle

const products = await Product.find().sort({ createdAt: -1 });
```

8.3 — Pagination (Page by Page)

Saare products ek saath nahi dikhte — page by page dikhte hain. `.skip()` aur `.limit()` se pagination hoti hai:

```
// Query parameters se lena: /api/products?page=2&limit=10

router.get('/', async (req, res) => {
  try {
    const page = Number(req.query.page) || 1; // Default: page 1
    const limit = Number(req.query.limit) || 10; // Default: 10 per page

    const skip = (page - 1) * limit;

    // Page 1: skip 0 | Page 2: skip 10 | Page 3: skip 20

    const total = await Product.countDocuments();
    const products = await Product.find()
      .skip(skip)
      .limit(limit)
      .sort({ createdAt: -1 });

    res.status(200).json({
      success: true,
      count: products.length,
      total: total,
      totalPages: Math.ceil(total / limit),
      currentPage: page,
      data: products
    });
  } catch (error) {
    res.status(500).json({ success: false, message: error.message });
  }
}
```

```
});
```

8.4 — Select, Populate aur Count

```
// Sirf specific fields select karo (baaki mat bhejo)
const products = await Product.find().select('name price images rating'
);
// Password automatically exclude hoga (select: false schema mein)

// Populate – ObjectId ki jagah actual data lao
// Product mein seller ka ObjectId hai – real user data chahiye
const product = await Product.findById(id).populate('seller', 'name email');
// seller field mein ab ObjectId nahi, User ka naam aur email hoga!

// Count karo – total kitne products hain?
const total = await Product.countDocuments();

const footwearCount = await Product.countDocuments({ category: 'Footwear' });

// Sab ek saath – filter + search + sort + paginate
const products = await Product
  .find({ category: 'Footwear', price: { $lte: 10000 } })
  .select('name price images rating')
  .sort({ rating: -1 })
  .skip(skip)
  .limit(limit)
  .populate('seller', 'name');
```

■ TIP

Queries chain kar sakte ho — `.find().sort().skip().limit().select().populate()` — ek ke baad ek lagao. Database tab query run karta hai jab `await` karo!

SECTION 9 — Summary, Practice & Next Steps

9.0 — Chapter 3 Ka Recap

■ MongoDB	NoSQL document database — JSON-like documents, flexible schema
■ MongoDB Atlas	Free cloud database — connection string se connect karo
■ .env + dotenv	Secrets protect karo — MONGO_URI kabhi code mein mat likho
■ Mongoose	ODM — MongoDB ko asaan banata hai — Schema, Model, Validation
■ Schema	Blueprint — fields, types, validation, defaults define karo
■ Model	Schema se banta hai — CRUD operations ke liye class
■ Product Model	name, price, images, category, stock, reviews, rating
■ User Model	name, email, password(hash), role, address, avatar
■ Order Model	user, orderItems, shippingAddress, totalPrice, status
■ .find() / findById()	READ — saare ya ek document lao
■ .create()	CREATE — naya document banana
■ .findByIdAndUpdate()	UPDATE — { new:true, runValidators:true }
■ .findByIdAndDelete()	DELETE — document permanently hatana
■ Validation	required, min, max, enum, unique, match, custom validators
■ Advanced Queries	.sort(), .skip(), .limit(), .select(), .populate()

9.1 — Practice Exercises

■ Easy Level

1. MongoDB Atlas account banao, free cluster setup karo
2. .env file banao — MONGO_URI aur PORT daalo
3. config/db.js banao aur server se connect karo

4. 'MongoDB Connected!' console mein dekho

■ Medium Level

1. Product model banao — saare fields ke saath
2. POST /api/products — real database mein product add karo
3. GET /api/products — database se saare products lao
4. Thunder Client se dono test karo — Atlas mein data dekho

■ Challenge Level

1. User model banao (password bcrypt ke bina abhi, sirf schema)
2. Order model banao — orderItems array ke saath
3. Products pe filtering add karo: ?category=Footwear
4. Pagination add karo: ?page=1&limit=5
5. .populate() use karo — seller ki info product ke saath
6. Validation errors properly handle karo

9.2 — Chapter 4 Ka Preview

REST API Architecture — Controllers aur Route Organization

- MVC pattern — Model, View, Controller architecture
- Controllers — route logic alag file mein
- Express Router — clean organized routes
- Async error handler middleware — DRY code
- ShopEase ka final backend structure
- Environment-based configuration
- API documentation — kaise likhte hain

Zabardast! Chapter 3 Complete! ■■

Tumne aaj real database connect kiya, 3 production-ready models banaye, CRUD operations kiye, validation seekhi, aur advanced queries chalayi. Yahi woh kaam hai jo real companies karte hain! **Chapter 4 mein hum code ko professional architecture mein organize karenge.**